

## Practical 6: Stack Implementation using Array and Linked List, with Expression Processing

### Practical Aim

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

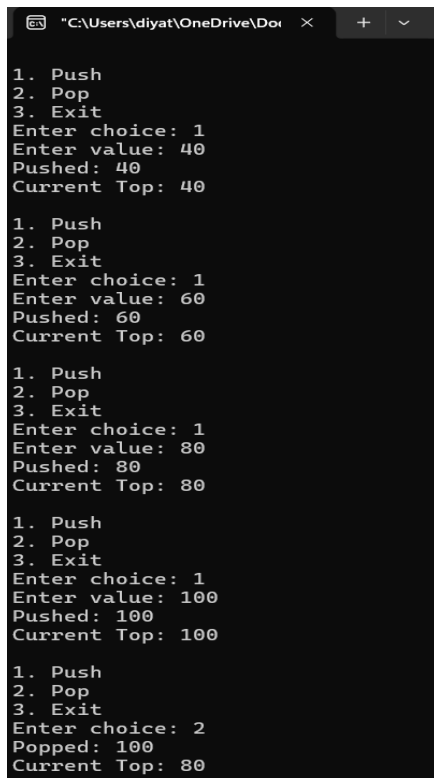
### Problem 1

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most  $n$  trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed. Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

### Source Code

[https://github.com/diyatolia1212/FDSA-25DCE115\\_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL%206](https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL%206)

### Output



```
"C:\Users\diyat\OneDrive\Docu... X + v
1. Push
2. Pop
3. Exit
Enter choice: 1
Enter value: 40
Pushed: 40
Current Top: 40

1. Push
2. Pop
3. Exit
Enter choice: 1
Enter value: 60
Pushed: 60
Current Top: 60

1. Push
2. Pop
3. Exit
Enter choice: 1
Enter value: 80
Pushed: 80
Current Top: 80

1. Push
2. Pop
3. Exit
Enter choice: 1
Enter value: 100
Pushed: 100
Current Top: 100

1. Push
2. Pop
3. Exit
Enter choice: 2
Popped: 100
Current Top: 80
```

## Conclusion

In this practical, we implemented a stack using an array. We learned how push and pop operations work using the LIFO principle. We also checked stack overflow when the stack is full and stack underflow when the stack is empty.

## Problem 2

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation. Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

## Source code

[https://github.com/diyatolia1212/FDSA-25DCE115\\_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL %206](https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL%206)

## Output

```
"C:\Users\diyat\OneDrive\Doi x + v
3. Exit
Enter choice: 1
Enter page name: google
Current Page: google

1. Visit Page
2. Back
3. Exit
Enter choice: 1
Enter page name: heollo
Current Page: heollo

1. Visit Page
2. Back
3. Exit
Enter choice: 1
Enter page name: hieeee
Current Page: hieeee

1. Visit Page
2. Back
3. Exit
Enter choice: 2
Going back from: hieeee
Current Page: heollo
```

## Conclusion

In this practical, we implemented a stack using a linked list. We learned how nodes can be added and removed dynamically. Unlike an array, the linked list does not have a fixed size, so memory is used according to the number of pages stored.

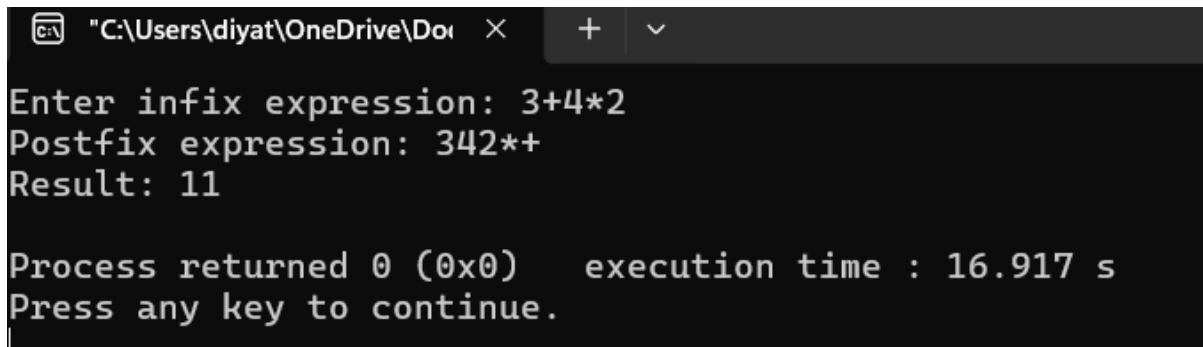
## Problem 3

A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g.  $3 + 4 * 2$  or  $(3 + 4) * 2$ ). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result. Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

## Source code

[https://github.com/diyatolia1212/FDSA-25DCE115\\_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL%206](https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/43e80b1aa7c70638c18f163d377eba0fc35a2139/PRACTICAL%206)

## Output



```
"C:\Users\diyat\OneDrive\Doi" X + v
Enter infix expression: 3+4*2
Postfix expression: 342*+
Result: 11

Process returned 0 (0x0)   execution time : 16.917 s
Press any key to continue.
```

## Conclusion

In this practical, we converted an infix expression into postfix using a stack. We learned how the stack helps to handle operators, priorities, and brackets. The postfix expression can then be evaluated easily without using brackets.

## Challenges Faced

Problem: Understanding when the stack is full or empty.

Solution: Used `top == 4` for overflow and `top == -1` for underflow.

Problem: Managing nodes while adding and removing pages.

Solution: Used top pointer to add and delete nodes correctly.

Problem: Handling operator priority and brackets correctly.

Solution: Used a stack to store operators and applied priority rules while converting.

## Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Answer: The front and rear positions can be moved back to 0 when they reach the last position. This can be done using % (modulo). The array is full when  $(\text{rear} + 1) \% \text{SIZE} == \text{front}$ .

2. [Problem 2] Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Answer: Yes. It can run out of memory when the computer has no more free memory available. The actual limit depends on the available RAM and system memory. This limit is not fixed in the code because nodes are created dynamically.

3. [Problem 3] operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Answer: The postfix expression may become incorrect because operators will not follow their proper order. For example, multiplication should be performed before addition. Operator precedence is needed to produce the correct postfix expression.